

PA1-2026

CS300 - CSC13002 - Introduction to Software Engineering

Project Assignment 1 (PA1-2026)

Total scores: 30 points

Duration: 2 weeks

This is a teamwork assignment.

A - Group Registration (0 points - administrative task)

You are asked to form groups of 3-5 students each and to register your group by filling in the registration form.

You need to specify:

- Your group name and group leader. The first person on the form is supposed to be the group leader.
- The student id, full name, and email address of each member in your group.
- The project name (can be changed later; however, changing your project after PA2 is not recommended as there will not be enough time to complete a new project).

Note: If a team member drops the course during the semester, the remaining members must continue with the project. Choose your teammates carefully and maintain a professional work ethic. A smaller team does not mean reduced expectations for the project's scope or quality.

Link of registration form: To be announced

Deadline: To be determined

B - Project Proposal (10 points)

Discuss within your group to propose a software project that your group will be working on. The project must be a **web application** or **mobile application** (desktop apps and games are temporarily not accepted). Your application should contain between 8 and 10 functional groups. A **functional group** is a cluster of related use cases that together serve a common purpose. For example: (1) authentication — sign up, log in, log out; (2) profile management — edit profile info, change password, change avatar; (3) search and browse — basic/advanced search, filter, sort results, view item details; (4) shopping cart — add to cart, edit cart, checkout. Focus on functional groups that are distinctive to your project idea rather than generic ones just to meet the quantity requirement.

Your apps should have at least 2 actors (2 types of users). Your apps should have at least 1 meaningful AI feature — the AI capability must provide real, practical value to the end user (e.g., chatbot-based search, AI-powered recommendations, vector search, intelligent content generation). Simple enhancements such as auto-suggesting search keywords do not qualify as a standalone AI feature.

Your apps should not violate Vietnamese laws, international regulations, or common ethical standards (e.g., gambling apps, pirated content streaming apps are not allowed).

Write a description of your project that includes the following information:

- **Introduction:** briefly describe your idea and explain its value.
- **Target users and environments:** who are your app's users, what environments they will use (devices, OS, Web, etc.)
- **Key features:** list the key features for the app. For each feature, write a short paragraph (2-3 sentences) describing what the feature does and why it is useful to the user. Do NOT just list feature names as bullet points.
- **AI feature:** propose at least 1 AI-powered feature for your app and describe how it enhances the user experience.

You are encouraged to consult the teaching assistants (TAs) for suggestions or feedback on your proposed project.

C - Existing App Survey (10 points)

Before building your own product, you must survey existing web/mobile apps that are similar to your proposed project. The goal is to understand what

solutions already exist on the market, learn from their user interface and workflow design, and identify opportunities for differentiation.

Note: As software engineering students, you are expected to be problem solvers, not just programmers. The quality of your project is your responsibility. The limitations of existing apps do not justify a low-quality project; rather, they represent opportunities for your team to innovate and improve upon.

Requirements:

- Identify at least 2 similar apps/websites currently available. If your project is highly niche and no directly comparable apps exist, you may survey apps that are closely related in domain or functionality.
- For each similar app, take screenshots covering the app's key features, user flows, and screens. The goal is to help your team thoroughly understand the product you want to build, so include enough screenshots to capture the essential aspects of each app.
- Organize the screenshots with captions explaining what each screen shows and how the feature works.
- Write a brief comparison summarizing:
 - What features these existing apps have in common.
 - What your proposed app will do differently or better.
 - Which UI/UX patterns you plan to adopt from these apps.

D - Team Contract (5 points)

Before starting projects, groups are asked to build their own team contract document. The following areas are suggested topics to cover. You may reorganize or combine sections to fit your team's actual situation:

1. **Team Roles and Responsibilities:** define each team member's role (e.g., Project Manager, Developer, Tester, etc.) and clearly specify each member's primary responsibilities and tasks. Due to the learning objectives of this course and the small team size, all students must act as full-stack engineers. The roles defined here are meant to set expectations for each team member. For example, if a student registers as the UI/UX designer, that student is expected to lead the UI/UX design phase, but other students should also participate in the design phase.

2. **Communication Plan:** outline communication tools (e.g., Slack, Discord, Zalo, etc.) and the frequency of meetings. Establish protocols for decision-making and conflict resolution. Set expectations for response times to messages and deliverables.
3. **Work Schedule and Deadlines:** agree on project milestones and deadlines. Specify availability of team members for meetings and work sessions. Include contingency plans for missed deadlines or unexpected delays.
4. **Code and Documentation Standards:** define coding conventions and tools. Outline expectations for code reviews and testing procedures. Include guidelines for project documentation.
5. **Accountability and Performance:** set criteria for measuring contribution and quality of work. Establish a process for handling underperformance or lack of participation. Include consequences for not adhering to the team contract.
6. **Decision-Making Process:** define how decisions will be made (e.g., majority vote, consensus). Outline who has the final say in case of disagreements.
7. **Conflict Resolution:** set a framework for resolving disputes between team members. Identify steps to escalate issues if necessary (e.g., involving a TA or instructor).
8. **Review and Update Process:** specify how and when the contract will be reviewed and updated. Ensure flexibility to make adjustments as the project progresses.

This contract will help set clear expectations and foster collaboration within the group.

E - Development Tools and Process Setup (5 points)

One of the main goals of this project assignment is to help you practice teamwork for professional software development. We will assess and evaluate your effectiveness in team collaboration. Everyone is required to work closely with others in the group to deliver results. We will use tools to monitor and assess each group's teamwork performance.

Process

Your team will follow the **Scrum** process throughout the project:

- Each PA corresponds to one Sprint (approximately 2-3 weeks).
- Each Sprint includes: 1 Sprint Planning meeting, 2 Scrum meetings (spread across the sprint), and 1 Sprint Review meeting.
- Follow the meeting format described in the "WeeklyReports" document (provided separately).

Required Tools

You are required to use the following tools:

1. **Moodle:** used for posting and submitting assignments.
2. **Facebook group (or equivalent):** used for general notifications, class discussions, and questions/answers.
3. **Communication tool** (Slack, Discord, Telegram, Messenger, Zalo, etc.): used for discussions and interactions among group members in the project. Each member is expected to check the communication app regularly.
4. **Jira** (or similar tools like Trello, Asana): used for task management following the Scrum model. **Important rules for task management:**
 - Every task related to the project must be logged: report writing tasks, coding tasks, self-training tasks, etc.
 - Each task must be assigned to exactly one team member (no shared tasks).
 - Each task must have dates for: **creation date**, **assignment date**, and **completion date**. The creation and assignment dates may be the same (e.g., when tasks are created and assigned during Sprint Planning), but the completion date must reflect when the work is actually finished.
 - **Do NOT** create a task, assign it, and complete it all at once after the work is already done. Tasks must be created and assigned **before** the work begins, then marked as done **when** the work is actually completed.
 - Groups must paste screenshots of all tasks in weekly reports.
5. **Git** (GitHub/GitLab/Bitbucket): used to store **both source code and documentation**. Since all reports and documentation will be written in text-based formats (Markdown), they should be version-controlled alongside the code. Your repository should be in **private mode** (the instructors/TAs

will request access when needed). **Important: Never commit API keys, secrets, or credentials to the repository to avoid security risks and potential financial loss.** The following is a suggested folder structure:

- `/src` : source code
- `/docs` : documentation
 - `management/` : planning documents, reports (weekly report, project status report, etc.)
 - `requirements/` : vision document, use cases
 - `analysis-and-design/` : software architecture document, diagrams, UI design
 - `test/` : test plan, test cases, test reports

6. **Spec Kit:** given that students are new to web/mobile app development and that AI-assisted development is now widely available, the instructors encourage all teams to adopt specification-driven development for building software products. Your team will use Spec Kit to manage the development workflow from specification to implementation. Detailed instructions on how to use Spec Kit will be provided separately. Spec Kit setup and initialization will be required starting from PA2.

7. **AI Coding Accounts:** each team member must register for free educational accounts on AI coding platforms (e.g., GitHub Copilot for Students, Cursor, etc.).

Notes on Curriculum Scope

This course focuses on the **software engineering process** (requirements, design, implementation workflow, testing, project management). The specific programming skills for web development, mobile development, game development, etc., are taught in subsequent courses in the Software Engineering program. In this course, you are expected to leverage AI coding tools and self-learning to implement your project.

With the increasing role of AI in software development, grading will consider not only your final deliverables but also your development progress. The instructors may use AI tools to summarize and review your work. Therefore, all submissions should include both text-based formats and PDF/image formats.

Submission Guidelines

- All documents must be written in **English** using **Markdown** format. Diagrams should be drawn using **Mermaid** syntax whenever possible.
- In each report/document section, the team must indicate who performed the work, who reviewed it, and who edited the section. Place this information on the first line immediately after the section header.
- Submit both the **original Markdown files (.md)** and **PDF files converted from Markdown**. You can use tools such as Notion or VS Code extensions that support Markdown with Mermaid rendering to export PDFs.
- Provide a **Git log** (screenshot or exported log) showing commit history of your repository.
- Compress all files into a directory named `PA1-Group[GroupId]` and submit as a zip file.
Example: `PA1-Group01.zip`

Grading Criteria

The assignment will be graded according to how well your team prepares the required documents. Criteria include:

- English writing: clear and easy to understand with few errors.
- Well-formatted texts and diagrams.
- Completeness and quality of the project proposal, existing app survey, and team contract.
- Proper setup and evidence of using required tools (Jira tasks screenshot, Git repository structure, AI coding accounts).